

PRN : 23610082

Name : Sanika Bakare

Title :

OpenMP Program for Matrix Addition Using Threads

Aim of Assignment:

To implement matrix addition using OpenMP threads in C/C++ and understand parallel execution for improving computation speed and efficiency.

H/W and S/W Required:

Hardware Requirements

Intel/AMD Processor

Minimum 4 GB RAM

Keyboard and Monitor

Software Requirements

Operating System: Windows/Linux

GCC Compiler with OpenMP support

Code Editor (VS Code / CodeBlocks / Dev C++)

Problem Statement

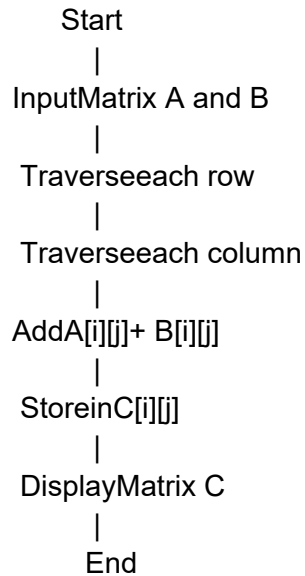
Write an OpenMP program to perform addition of two matrices using parallel threads. The program should divide the work among multiple threads so that matrix elements are added simultaneously.

Sequential Solution Logic with Diagram

Sequential Logic

- 1) Start the program.
- 2) Declare two matrices A and B.
- 3) Input matrix elements.
- 4) Traverse matrix using nested loops.
- 5) Add corresponding elements:
- 6) Display resultant matrix.
- 7) Stop.

Sequential Flow Diagram



Parallel Approach with Diagram

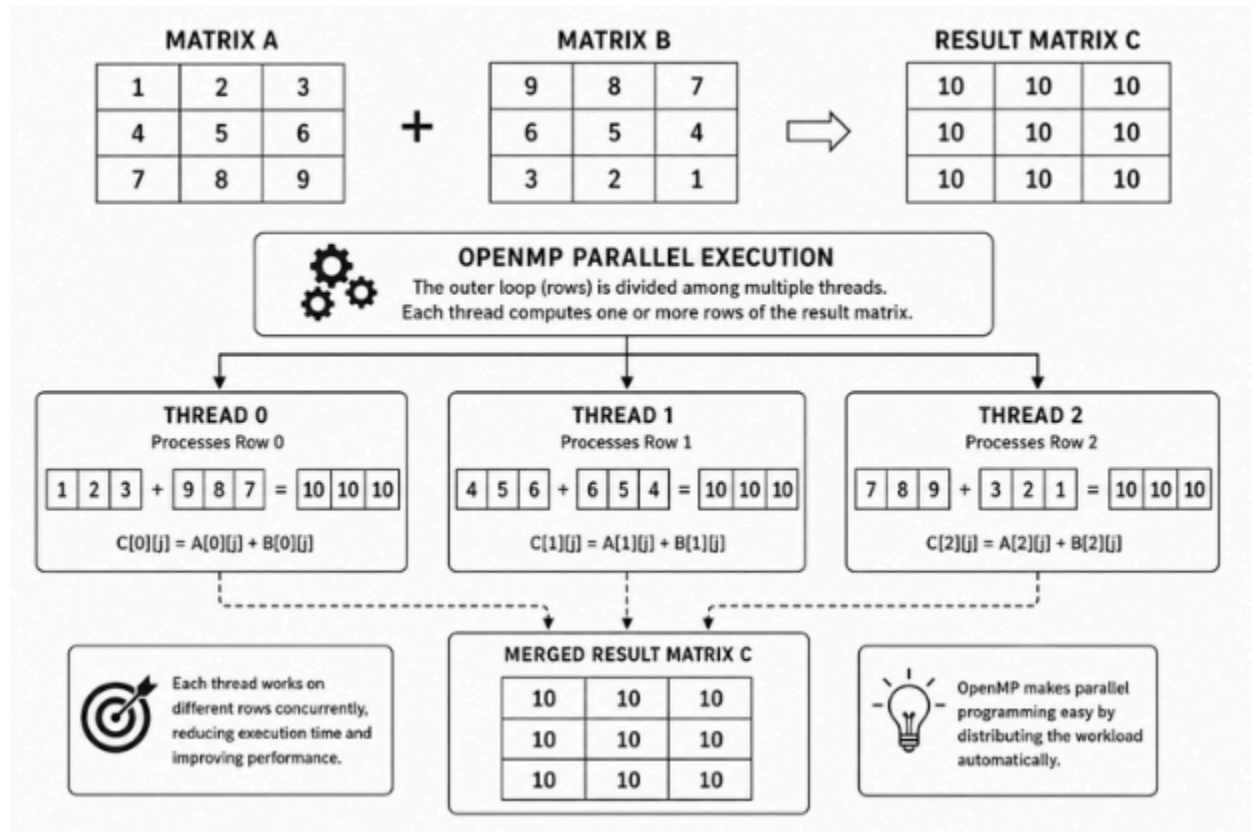
Parallel Approach

In OpenMP, multiple threads execute matrix addition simultaneously.

- 1) Different rows/elements are divided among threads.
- 2) Each thread independently computes part of the resultant matrix.
- 3) Parallel execution reduces execution time.

OpenMP uses:

`#pragma omp parallel for`
to distribute loop iterations among threads



Code with Explanation

OpenMP Program for Matrix Additionode with Explanation

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int a[3][3], b[3][3], c[3][3];
    int i, j;

    printf("Enter elements of Matrix A:\n");
    for(i = 0; i < 3; i++)
    {
        for(j = 0; j < 3; j++)
        {
            scanf("%d", &a[i][j]);
```

```

    }
}

printf("Enter elements of Matrix B:\n");
for(i = 0; i < 3; i++)
{
    for(j=0;j<3;j++)
    {
        scanf("%d",&b[i][j]);
    }
}

#pragma omp parallel for private(j)
for(i = 0; i < 3; i++)
{
    for(j=0;j<3;j++)
    {
        c[i][j]=a[i][j]+b[i][j];

        printf("Thread%d computed c[%d][%d]\n",
            omp_get_thread_num(), i, j);
    }
}

printf("\nResultantMatrix:\n");

for(i = 0; i < 3; i++)
{
    for(j=0;j<3;j++)
    {
        printf("%d",c[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Explanation of Program

This program performs Matrix Addition using OpenMP threads on a multicore CPU. Matrix addition is an operation in which corresponding elements of two matrices are added to generate a resultant matrix.

Matrix Addition Formula

$$C[i][j] = A[i][j] + B[i][j]$$

Where:

$A[i][j]$ = element of first matrix

$B[i][j]$ = element of second matrix

$C[i][j]$ = resultant matrix element

The program declares three matrices:

Matrix A stores elements of the first matrix.

Matrix B stores elements of the second matrix.

Matrix C stores the resultant matrix after addition.

The user enters elements of matrices A and B using nested loops. The outer loop traverses rows, while the inner loop traverses columns of the matrices.

Matrix A:

1 2 3

4 5 6

7 8 9

Matrix B:

9 8 7

6 5 4

3 2 1

The addition operation is performed as:

$$C[0][0] = 1 + 9 = 10$$

$$C[0][1] = 2 + 8 = 10$$

$$C[0][2] = 3 + 7 = 10$$

$$C[1][0] = 4 + 6 = 10$$

$$C[1][1] = 5 + 5 = 10$$

$$C[1][2] = 6 + 4 = 10$$

$$C[2][0] = 7 + 3 = 10$$

$$C[2][1] = 8 + 2 = 10$$

$$C[2][2] = 9 + 1 = 10$$

The OpenMP directive:
#pragma omp parallel for

is applied before the outer loop. This directive creates multiple threads and automatically distributes loop iterations among them. Each thread processes different rows of the matrix simultaneously, enabling parallel execution.

For example:

Thread 0 may process Row 0

Thread 1 may process Row 1

Thread 2 may process Row 2

The statement:

$c[i][j] = a[i][j] + b[i][j];$

adds corresponding elements of matrices A and B and stores the result in matrix C.

The function:

omp_get_thread_num()

returns the ID of the thread currently executing the operation. This helps in observing how multiple threads work concurrently during execution.

After all threads complete execution, the resultant matrix is displayed using nested loops.

Final Output:

```
10 10 10
10 10 10
10 10 10
```

The program demonstrates how OpenMP improves computational efficiency by executing matrix operations concurrently instead of sequentially. Parallel execution reduces computation time and utilizes multicore processors effectively, especially for large matrices.

Output :

```
raj@rajgit1:~/libmad-0.10: $ sudo apt update
[sudo] password for raj:
Hit:1 https://brave-browser-apt-release.s3.brave.com stable InRelease
Hit:2 https://download.docker.com/linux/ubuntu jammy InRelease
Hit:3 https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/6.0 InRelease
Hit:4 https://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:5 https://it.archive.ubuntu.com/ubuntu jammy InRelease
Hit:6 https://it.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 https://it.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:8 https://ppa.launchpadcontent.net/ser-priog/stable/ubuntu jammy InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
122 packages can be upgraded. Run 'apt list --upgradable' to see them.
... Skipping acquire of configured file 'main/binary-libs/packages' as repository 'https://brave-browser-apt-release.s3.brave.com stable InRelease' doesn't support architecture 'i386'
... Skipping acquire of configured file 'stable/binary-libs/packages' as repository 'https://download.docker.com/linux/ubuntu jammy InRelease' doesn't support architecture 'i386'
... https://repo.mongodb.org/apt/ubuntu/dists/jammy/mongodb-org/6.0/InRelease: Key is stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATION section in apt-key(8) for details.
raj@rajgit1:~/libmad-0.10: $ sudo apt install gcc
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gcc is already the newest version (4:11.2.0-1ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 122 not upgraded.
raj@rajgit1:~/libmad-0.10: $
```

```

#include <stdio.h>
#include <omp.h>

int main()
{
    int a[10][10], b[10][10], c[10][10];
    int i, j, r, col;

    printf("Enter number of rows and columns: ");
    scanf("%d%d", &r, &col);

    printf("Enter elements of Matrix A:\n");
    for(i = 0; i < r; i++)
    {
        for(j = 0; j < col; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Enter elements of Matrix B:\n");
    for(i = 0; i < r; i++)
    {
        for(j = 0; j < col; j++)
        {
            scanf("%d", &b[i][j]);
        }
    }

    #pragma omp parallel for private(j)
    for(i = 0; i < r; i++)
    {
        for(j = 0; j < col; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    }

    printf("Resultant Matrix:\n");
    for(i = 0; i < r; i++)
    {
        for(j = 0; j < col; j++)
        {
            printf("%d ", c[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

```

ragini@ragini-ThinkPad-E15:~$ gcc -fopenmp matrix_addition.c -o matrix
ragini@ragini-ThinkPad-E15:~$ ./matrix
Enter number of rows and columns: 8 8
Enter elements of Matrix A:
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16
17 18 19 20 21 22 23 24
25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48
49 50 51 52 53 54 55 56
57 58 59 60 61 62 63 64
Enter elements of Matrix B:
64 63 62 61 60 59 58 57
56 55 54 53 52 51 50 49
48 47 46 45 44 43 42 41
40 39 38 37 36 35 34 33
32 31 30 29 28 27 26 25
24 23 22 21 20 19 18 17
16 15 14 13 12 11 10 9
8 7 6 5 4 3 2 1
Resultant Matrix:
65 65 65 65 65 65 65 65
65 65 65 65 65 65 65 65
65 65 65 65 65 65 65 65
65 65 65 65 65 65 65 65
65 65 65 65 65 65 65 65
65 65 65 65 65 65 65 65
65 65 65 65 65 65 65 65
65 65 65 65 65 65 65 65
ragini@ragini-ThinkPad-E15:~$ █

```

Conclusion :

The OpenMP program for Matrix Addition was successfully implemented using parallel threads. The program added corresponding elements of two matrices and generated the resultant matrix efficiently. OpenMP enabled parallel execution of matrix operations, improving CPU utilization and reducing execution time compared to sequential execution.